SOAR Lite - Simple Project Explanation

Prepared for presentation use

1. What this project is

SOAR Lite is a mock, demo-only Security Orchestration, Automation and Response (SOAR) project. It is not connected to real firewalls, identity systems, servers, or network devices. Instead, it simulates what a response team would do during an incident, but only inside memory and in the browser/app session.

The project teaches the core idea behind SOAR: receive an alert, evaluate the impact, create a playbook action, require human approval, and keep an audit trail of every step. It is built for learning, demonstration, and portfolio work, not for production operations.

2. Main idea of the system

The system has two main parts: a backend API and a frontend/demo UI. The backend stores actions in memory and decides whether they are pending, approved, rejected, or rolled back. The frontend shows these actions in a simple control panel and allows users to approve or reject them.

There are three mock response playbooks:

- Host/IP isolation: simulate quarantining a host.
- Session revocation: simulate revoking a user session.
- Domain/URL block: simulate creating a deny list rule.

3. How the backend works

When a user creates a new action, the workflow happens in this order:

- A request is received from the UI or API client.
- The target playbook is selected, such as session_revocation or domain_block.
- The service calculates an impact simulation and risk level.
- The request is set to pending_human_approval and is not executed automatically.
- A human reviewer approves or rejects the action.
- If approved, it is marked as executed_mock.
- Optional rollback is simulated only in memory and does not touch real systems.

This means the project is designed to teach approval-gated response without creating real operational risk.

4. File-by-file explanation

- README.md

This is the project guide. It explains what SOAR Lite is, the synthetic safety rules, the demo workflow, how to run the app, and how the project is meant to be used. It is the easiest file to understand before reading the code.

- main.py

This is the root startup file. It imports the FastAPI app from the package and runs it through uvicorn. It is the entry point for the backend server.

- streamlit_app.py

This is the web UI. It creates the dashboard, loads demo scenarios, shows approval actions, shows the audit history, and includes the threat checker tab. It has the buttons and tabs users interact with during the presentation.

- soar_lite/__init__.py

This file marks the package and provides the top-level package identity. It is minimal and mostly used as a package initializer.

- soar_lite/models.py

This contains all data structures. It defines the available playbook names, action status values, impact simulation data, request/response models, and audit event model. It is the schema for the entire backend.

- soar_lite/playbooks.py

This file defines the three mock playbook simulations. Each playbook returns a description of the expected effect and the side effects, plus a risk level. It does not perform any real action.

- soar_lite/service.py

This is the main logic engine. It creates, stores, lists, approves, and rejects actions. It also handles rollback simulation. This is the heart of the workflow logic.

- soar_lite/audit.py

This keeps an in-memory log of all actions and workflow changes. Every time a request is received, approved, rejected, or rolled back, an event is recorded here.

- soar_lite/main.py

This is the actual FastAPI application. It exposes endpoints such as /health, /playbooks, /actions, /actions/{id}/decision, and /audit. These endpoints form the API backend.

- soar_lite/inspector.py

This file performs local, heuristic inspection of suspicious text, URLs, email content, PDFs, QR/barcodes, and phone numbers. It does not connect to external threat feeds or reputation databases. It is a safety demo feature for checking indicators without contacting the internet.

- tests/test_api.py

This contains automated tests that verify the approval gate. It proves actions are created as pending, that human approval changes the state, that rejection is forbidden after approval, and that rollback only works after mock execution.

- requirements.txt

This lists all required packages for FastAPI, Streamlit, Pydantic, PDF processing, QR decoding, and testing.

5. Important backend models

The most important model is ContainmentAction, which carries the action lifecycle. It includes keys like action_id, playbook, target, rationale, requested_by, status, impact_simulation, created_at, updated_at, and

optional decision information.

The request model ActionRequest lets the client send: playbook, target, rationale, and requested_by. The approval model ApprovalDecision allows a reviewer to either approve or reject the action with a reason. The ImpactSimulation model describes the fake effect and risk.

The status values are:

- pending_human_approval: request created and waiting for decision
- rejected: reviewer declined the action
- executed_mock: approved, but only in memory
- rollback_simulated: the mock rollback step recorded

6. The three playbooks in detail

Host/IP isolation: The system creates a simulation saying a host would be quarantined. It affects the target resource and warns that access to services could be lost if this were real. The risk is HIGH.

Session revocation: The system creates a simulation for revoking active sessions. It warns the user may need to log in again. The risk is MEDIUM.

Domain/URL block: The system creates a simulation for a deny rule against a domain or URL. It warns legitimate traffic may be blocked if the indicator was wrong. The risk is MEDIUM.

7. Why the approval gate matters

This is the most important learning point of the project. The response is never executed automatically. Every action waits for explicit review. That means the project demonstrates safe workflow design: inspect, estimate effect, ask for approval, then allow a mock execution only after a human makes the decision.

Without the approval gate, a security tool could make dangerous changes. SOAR Lite teaches the safe pattern instead of the unsafe pattern.

8. How the Streamlit app works

When the app starts, it creates an Orchestrator object and stores it in Streamlit session state. That object keeps all actions in memory for that session. The app then displays a dashboard with counts for actions, pending items, executed items, and audit events.

The tabs are:

- Start here: guided demo of a suspicious sign-in incident.
- Threat checker: inspect text, URLs, emails, PDFs, QR images, and phone numbers.
- New simulation: create a custom mock playbook action.
- Approval queue: review all pending actions and approve or reject them.
- Audit trail: view structured lifecycle events for each action.

9. How the threat checker works

The inspector file reads text locally and looks for suspicious patterns. It extracts URLs, checks for shorteners, IP addresses, punycode, odd domains, credential-themed terms, and social engineering language. It also looks for email headers, phone numbers, and decoded text from QR/barcodes or PDFs.

Important safety rule: the checker does not open the destination URL, call external APIs, or query a reputation service. It only produces heuristic warnings, not a confirmed trust verdict.

10. Flow diagram in plain English

User/analyst -> Creates action request -> Impact simulation -> Action enters pending approval -> Human reviewer chooses approve or reject -> If approved, status becomes executed_mock -> Optional rollback simulation -> Audit trail records all events.

11. What is real vs fake in this system

Real: the app runs in Python, stores data in memory, and uses FastAPI and Streamlit. Fake: no actual firewall rules are changed, no real accounts are modified, no email or network APIs are called, no real identity system is touched, no real database is used. Everything is in-memory and demo-only.

12. Why this project matters

This project teaches the structure of a basic SOC or SOAR workflow. It is the best way to explain the concept of response orchestration to beginners because it is small and understandable. The project makes security workflow design easy to explain: simulate, review, approve, log, and rollback only in-memory.

13. How to run it

From the project root, create a virtual environment and install dependencies. Then run the API and the Streamlit UI. The API is typically served with uvicorn and the UI with streamlit run streamlit_app.py.

The project is designed to be run locally for demonstrations, not deployed to a real security environment. The README.md already contains the exact setup steps.

14. Final summary

SOAR Lite is a learning project that simulates a safe incident response workflow. It covers mock actions, approval gates, risk scoring, audit logging, and local suspicious-content inspection. It is intentionally simple, but it demonstrates the same ideas used by real SOAR systems: investigate, assess, confirm, approve, execute, and record.

This guide is intentionally simple and presentation-ready. Use it to explain the project in short, plain English.